ONLINE MASTERS IN **DATA SCIENCE**

DSC258: NATURAL LANGUAGE PROCESSING

Interesting Properties/ Applying Word Embeddings

UC San Diego

COMPUTER SCIENCE & ENGINEERING

HALICIOĞLU DATA SCIENCE INSTITUTE

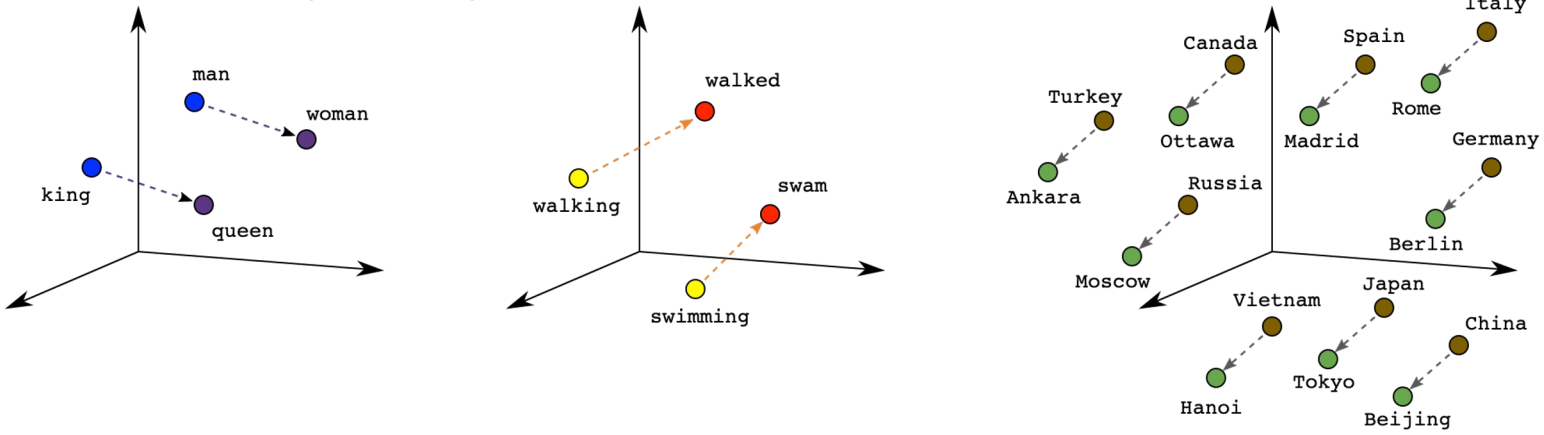
OUTLINE

Word Embedding

- Traditional Distributional Representation
- Dense Vector Representations
- **Interesting Properties of Word Vectors**
- Applying Word Embeddings

Interesting properties of the word vectors

- Word vectors capture many linguistic regularities
- Famous example
  - vector('king') - vector('man') + vector('woman') is close to vector('queen')



Male-Female Verb Tense Country-Capital

https://developers.google.com/machine-learning/crash-course/embeddings/translating-to-a-lower-dimensional-space

OUTLINE

Word Embedding

- Traditional Distributional Representation
- Dense Vector Representations
- Interesting Properties of Word Vectors
- **Applying Word Embeddings**

Popular Pre-trained Word Embedding

GoogleNews-vectors-negative300.bin.gz

- 300 dimensions
- 3 million words and phrases
- Trained on news articles (~100 billion words!)

Glove 6B

- Many dimension choices (e.g., 100)
- 400K vocab (uncased)
- Trained on Wikipedia 2014 + Gigaword 5 (6B tokens)

Train Your Own Embedding

For domain specific applications, embedding trained in the same domain will be very helpful

Python package

- gensim
- fasttext
- …

What you need is just a large enough raw corpus

You can initialize the vectors by some pre-trained ones

Coding time!

Let’s try out word embedding together

CODING TIME !

In [1]:
```
import gensim.downloader as api
wv = api.load('word2vec-google-news-300') # the download could take you some time
```

In [3]:
```
vec_king = wv['king']
print(len(vec_king), vec_king)
```

```
300 [ 1.25976562e-01  2.97851562e-02  8.60595703e-03  1.39648438e-01
 -2.56347656e-02 -3.61328125e-02  1.11816406e-01 -1.98242188e-01
  5.12695312e-02  3.63281250e-01 -2.42187500e-01 -3.02734375e-01
 -1.77734375e-01 -2.49023438e-02 -1.67968750e-01 -1.69921875e-01
  3.46679688e-02  5.21850586e-03  4.63867188e-02  1.28906250e-01
  1.36718750e-01  1.12792969e-01  5.95703125e-02  1.36718750e-01
  1.01074219e-01 -1.76757812e-01 -2.51953125e-01  5.98144531e-02
  3.41796875e-01 -3.11279297e-02  1.04492188e-01  6.17675781e-02
  1.24511719e-01  4.00390625e-01 -3.22265625e-01  8.39843750e-02
  3.90625000e-02  5.85937500e-03  7.03125000e-02  1.72851562e-01
  1.38671875e-01 -2.31445312e-01  2.83203125e-01  1.42578125e-01
  3.41796875e-01 -2.39257812e-02 -1.09863281e-01  3.32031250e-02
 -5.46875000e-02  1.53198242e-02 -1.62109375e-01  1.58203125e-01
 -2.59765625e-01  2.01416016e-02 -1.63085938e-01  1.35803223e-03
 -1.44531250e-01 -5.68847656e-02  4.29687500e-02 -2.46582031e-02
  1.85546875e-01  4.47265625e-01  9.58251953e-03  1.31835938e-01
  9.86328125e-02 -1.85546875e-01 -1.00097656e-01 -1.33789062e-01
 -1.25000000e-01  2.83203125e-01  1.23046875e-01  5.32226562e-02
 -1.77734375e-01  8.59375000e-02 -2.18505859e-02  2.05078125e-02
 -1.39648438e-01  2.51464844e-02  1.38671875e-01 -1.05468750e-01
  1.38671875e-01  8.88671875e-02 -7.51953125e-02 -2.13623047e-02
```

CODING TIME !

In [6]:
```
try:
    vec_cameroon = wv['cameroon']
except KeyError:
    print("The word 'cameroon' does not appear in this model")
```

```
The word 'cameroon' does not appear in this model
```

In [9]:
```
pairs = [
    ('car', 'minivan'),    # a minivan is a kind of car
    ('car', 'bicycle'),    # still a wheeled vehicle
    ('car', 'airplane'),   # ok, no wheels, but still a vehicle
    ('car', 'cereal'),     # ... and so on
    ('car', 'communism'),
]
for w1, w2 in pairs:
    sim = wv.similarity(w1, w2)
    print('%r\t%r\t%.2f' % (w1, w2, sim))
```

```
'car'	'minivan'	0.69
'car'	'bicycle'	0.54
'car'	'airplane'	0.42
'car'	'cereal'	0.14
'car'	'communism'	0.06
```

CODING TIME !

Nearest Neighbor Search

- Most similar words based on word embedding vectors

In [10]:
```
print(wv.most_similar(positive=['car'], topn=20))
```

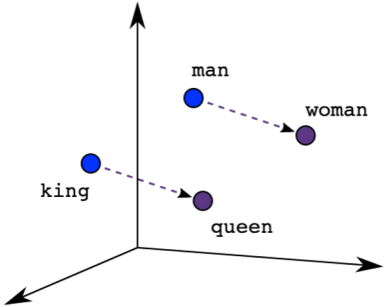
```
[('vehicle', 0.7821096181869507), ('cars', 0.7423830032348633), ('SUV', 0.7160962820053101), ('minivan', 0.6907036304
473877), ('truck', 0.6735789775848389), ('Car', 0.6677608489990234), ('Ford_Focus', 0.667320191860199), ('Honda_Civi
c', 0.662684977054596), ('Jeep', 0.6511331796646118), ('pickup_truck', 0.64414381980896), ('Toyota_Camry', 0.63951730
72814941), ('scooter', 0.638153076171875), ('Honda_Accord', 0.6343971490859985), ('sedan', 0.6336702108383179), ('Toy
ota_Corolla', 0.6304991841316223), ('motorcycle', 0.6256054043769836), ('Nissan_Altima', 0.6253108978271484), ('Ford_
Explorer', 0.6211567521095276), ('Ford_Escort', 0.6211459040641785), ('Ford_Mustang', 0.6198910474777222)]
```

Find the outlier in a list of words

In [28]:
```
print(wv.doesnt_match(['fire', 'water', 'land', 'sea', 'air', 'car']))
```

```
car
```

CODING TIME !



Male-Female

```
In [27]: woman = wv['queen'] - wv['king'] + wv['man']
         real_woman = wv['woman']
         print( dot(woman, real_woman)/(norm(woman)*norm(real_woman)) )

         0.7186802
```

OUTLINE

Word Embedding

- Traditional Distributional Representation
- Dense Vector Representations
- Interesting Properties of Word Vectors
- Applying Word Embeddings